

Technical Evaluation Report

Task 2: Feature Engineering, Model Optimization & Performance Comparison

Prepared For: Maincrafts Technology Internship Portfolio

Date: June 2026

System Focus: Enhanced House Price Prediction System

Status: Completed Deliverables

Section 1: Executive Summary & Objectives

In professional data science environments, machine learning workflows go far beyond basic model training. Initial modeling attempts rarely produce production-ready systems, and models are rarely deployed after the first configuration pass. Instead, data is cleaned, transformed, and refined iteratively, and different algorithmic rules must be evaluated to execute strategic choices using explicit, measurable performance metrics.

This technical report details the complete development lifecycle of an enhanced House Price Prediction system built using the California Housing Dataset. The core execution pipeline integrates automated feature engineering, systematic matrix scaling, multi-model evaluation frameworks, and objective statistical validation to confidently isolate the optimal deployment candidate.

Core Engineering Objectives

- **Data Preparation:** Standardize and clean multidimensional array vectors to prevent numeric scale imbalances.
- **Feature Optimization:** Apply zero-mean, unit-variance vector scaling rules to improve gradient descent stability.
- **Algorithmic Diversity:** Simultaneously train linear, regularized, and hierarchical partition configurations to capture structural boundaries.
- **Statistical Validation:** Evaluate predictions against validation metrics to select the most accurate system.

Section 2: Dataset Characteristics & Target Variables

The system utilizes the **California Housing Dataset**, an industry benchmark for evaluating complex regression frameworks. The configuration encompasses authentic demographic boundaries and regional housing values mapped out across structural geographic coordinates.

Feature Definitions & Structural Layout

To construct the execution pipeline, properties are split into structured target parameters and objective input arrays:

- **Target Variable (y):** HousePrice / Represents the Median House Value expressed across block metrics (in hundreds of thousands of dollars).
- **Median Income:** Independent feature quantifying aggregate block salary distributions; typically demonstrates the highest linear correlation with housing valuation scales.

- **House Age & Average Rooms:** Core structural indicators that define regional real estate characteristics and property size averages.
- **Demographic Matrices:** Total population clusters, total household footprints, and precise latitude/longitude coordinates to model geographic dependencies.

Section 3: Preprocessing & Implementation Methodology

Raw input features contain massive numeric scale discrepancies (e.g., median income ranges below 15, while total population elements scale into the thousands). Features with naturally larger absolute scales can mathematically dominate variance computations, making training unstable.

Feature Scaling (Standardization)

To fix scale discrepancies, the system implements a StandardScaler transformation. This tool evaluates the column mean (μ) and standard deviation (σ) to map features uniformly onto a standardized normal profile where each variable exhibits a zero mean and unit variance:

Standardization Formula:

$$\begin{aligned}\mu &= (1 / n) * \sum x_i \\ \sigma &= \sqrt{[(1 / n) * \sum (x_i - \mu)^2]} \\ z &= (x - \mu) / \sigma\end{aligned}$$

Following feature transformation, the data matrix is split using an **80/20 train-test ratio**. The partitioning seed is locked using `random_state=42` to ensure identical testing splits across future model iterations.

Section 4: Source Pipeline Implementation

The clean, executable pipeline is written in Python using optimized Scikit-Learn libraries:

```
# Import essential core modules and model architectures
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Fetch, isolate, and scale the structural boundaries
data = fetch_california_housing(as_frame=True)
df = pd.concat([data.data, data.target.rename('HousePrice')], axis=1)
X = df.drop('HousePrice', axis=1)
y = df['HousePrice']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Segment rows into identical train/test partitions
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

Section 5: Performance Evaluation & Metrics

To ensure robust evaluation, performance is analyzed across two primary metrics: **Root Mean Squared Error (RMSE)** to measure absolute prediction accuracy, and the **Coefficient of Determination (R^2)** to measure how well the model accounts for variance.

Model Evaluation Architecture	RMSE Matrix Outcome ↓	R ² Score Metric ↑
Linear Regression Baseline	0.7456	0.5758
Ridge Regression (Regularized, $\alpha=1.0$)	0.7456	0.5758
Decision Tree Regressor (max_depth=5)	0.7301	0.5932

Section 6: Key Findings & Final Selection

Linear vs. Regularized Frameworks: Both standard Linear Regression and L_2 -regularized Ridge Regression achieved identical scores to four decimal places (RMSE: 0.7456, R^2 : 0.5758). This demonstrates that default regularization adjustments ($\alpha=1.0$) do not change the coefficients significantly on this scaled dataset.

Non-Linear Advantages: The Decision Tree Regressor (max_depth=5) achieved the best performance, lowering the error margin to **0.7301** and increasing variance coverage to **0.5932**.

Final Model Selection & Justification

The **Decision Tree Regressor (max_depth=5)** is selected as the optimal model for final system deployment. Real-world housing prices are heavily influenced by non-linear trends—such as geographic locations and localized income thresholds—which are difficult for simple straight-line equations to capture. By splitting data into hierarchical segments, the Decision Tree successfully uncovers these non-linear patterns, delivering higher predictive accuracy.

Section 7: Professional Takeaways & Workflow Validation

- Completing this structured assignment demonstrates several core skills essential to professional machine learning workflows:
- Data Preprocessing:** Successfully applied preprocessing transformations to prepare raw metrics for modeling.
 - Standardization Protocol:** Demonstrated how feature scaling keeps models stable and prevents wide numerical ranges from skewing predictions.
 - Objective Evaluation:** Established an evaluation workflow to compare multiple algorithms side-by-side using standardized statistical metrics.
 - Justified Decisions:** Applied data-driven performance comparisons to confidently select and explain the best model for final deployment.